sas innovate 2026

Data Driven Programming with CAS in SAS® Viya®

Hands On Workshop

Stephen Foerster
Data Engineering and Decisioning Lead, SAS Education

Copyright © SAS Institute Inc. All rights reserved.

Agenda

Data Driven Programming with CAS

Practical Examples

Compare to Base SAS

Compare to SAS/Macro

1. Introduction to CASL Variables
2. CASL Variables, more Detail
3. CAS Actions, Briefly
4. CAS Action Result Variables
5. Load All CSV Files in a Directory to CAS
6. Drive Processing from a Table

1. Introduction to CASL Variables

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

Introduction to CASL Variables

SAS Macro vs CASL

CASL Variable

```
%macro conLogic;
  %let aggreg8=N;

  %if &aggreg8=Y %then %do;
    Data public.Agg_Profit ;
    Set public.Orion_Profit ;
      keep Product_Category Customer_Group aggProfit;
      by Product_Category Customer_Group;
      retain aggProfit;
      if first.Customer_Group then AggProfit = 0;
      aggProfit = aggProfit + Profit;
      if last.Customer_Group then output;
    run;


    Proc Print data=public.Agg_Profit;
    run;
  %end;
  %else %do;
    Proc Print data=public.Orion_Profit;
    run;
  %end;
%mend;

%conLogic;
```

```
Proc CAS;
  aggreg8='N';

  if aggreg8='Y' then do;
    datastep.runCode submit / code="
      Data public.Agg_Profit (replace=yes) ;
      Set public.Orion_Profit ;
           keep Product_Category Customer_Group aggProfit;
           by Product_Category Customer_Group;
           retain aggProfit;
           if first.Customer_Group then AggProfit = 0;
           aggProfit = aggProfit + Profit;
           if last.Customer_Group then output;
      run;";

      table.fetch / to=5000
           table={caslib="public" name="aggregatedTable"};
  end;
  else do;
      table.fetch / to=5000
           table={caslib="public" name="detailedTable"};
  end;
quit;
```

CASL variables are comparable to SAS Macro variables.
The comparison shows a CASL variable and macro variable, aggreg8, serving the same purpose.
If yes, the data is aggregated before printing. If not, the detailed data is printed.

2. CASL Variables, More Detail

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

CASL Variables, More Detail

Basic Types

```
Proc CAS;
    /* String */
    custName = "Mary Smith";
    print custName " likes SAS";
    /* Numeric  */
    y = 99;
    print "y plus 50 = " y + 50;
    /* Boolean  */
    exists = true;
    if exists then print exists;
    /* String array */
    colNames = {"make", "model", "type"};
    print "The 2nd column is " colNames[2];
    /* Numeric array */
    percentiles = {5, 25, 50, 75, 95};
    percentiles = percentiles + 99;
    print "The report shows: " percentiles[1:6];
    /* Multi-dimensional array */
    multiDim = { {1, 2}, {3, 4}, {5, 6} };
    print "The 2nd unit of the 3rd pair is " multiDim[3,2];
run;
```

SAS Log

```
Mary Smith likes SAS

y plus 50 = 149

TRUE

The 2nd column is model

The report shows: {5,25,50,75,95,99}

The 2nd unit of the 3rd pair is 6
```

Unlike macro variables which are just text, CASL variables can be strings, numeric, Boolean, as well as arrays.
The examples show the different data types as well as some minor manipulation, printing, and referencing techniques.

CASL Variables, More Detail

Basic Dictionary Variables

```
proc cas;
    /* Create a dictionary variable */
    myTable = {name="cars", caslib="casuser", replace=true};

    /* Display the structure in the log */
    describe myTable;

    /* Access with an individual key */
    print "Table name is " myTable.name;
quit;
```

SAS Log

```
dictionary ( 3 entries, 3 used);
 [name] string;
 [caslib] string;
 [replace] boolean;

Table name is cars
```

CASL variables can also be dictionaries.
Dictionaries are like arrays except each position is named.
First we see a dictionary, myTable with three entries, name, caslib, and replace.
Each entry can be string, number, Boolean, etc.
To see the structure of a dictionary use the describe statement.
To reference an entry, use the dictionary name, “dot,” entry notation.
For example, myTable.name.

CASL Variables, More Detail

Nested Dictionary Variables

```
proc cas;
    /* Create a dictionary variable */
    myCar = {name="Odyssey", Features = {wheels=4, seats=7, color="red"}};

    /* Display the structure in the log */
    describe myCar;

    /* Access with an individual key */
    print myCar.Features.color;
quit;
```

SAS Log

```
dictionary ( 2 entries, 2 used);
 [name] string;
 [Features] dictionary ( 3 entries, 3 used);
  [wheels] int64_t;
  [seats] int64_t;
  [color] string;


red
```

Dictionaries can also contain other dictionaries.
Here we see a dictionary, myCar, that contains a string and another dictionary named Features.
The Features dictionary contains two numeric entries and a string entry.
To reference entries in nested dictionaries, use the dictionary.nested dictionary.entry notation.
We discuss nesting here because it will be important later.

3. CAS Actions

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

CAS Actions

Briefly

dataStep.**runCode**

simple.**Summary**

regression.**GLM**

table.**loadTable**

table.**tableInfo**

table.**fileInfo**

- Run Inside PROC CAS
- Fine Grain Processing Tools
- Combine with Logic, Variables, and other Actions

To achieve data driven processing with CASL, we need to understand CASL variables as well as CAS actions.
CAS actions are like miniature SAS procedures (PROCs).
The runCode action was shown earlier and it runs DATA Step code in CAS.
The Summary action acts like PROC Summary.
GLM runs a linear regression in CAS.
They are fine grain and meant to do only one action where a PROC may perform multiple actions.
They are meant to be used like methods, combined with logic and other CAS actions to build complex processing.
We'll use them this way, later in the presentation.

CAS Actions

Examples

```
/* Run the FileInfo action */
proc cas;
    table.fileinfo / caslib="public"
quit;
```

Results from table.fileInfo

File Information for root of caslib Public.

Permission	Owner	Group	Name	Size of File in Bytes
-rwxrwxrwx	sas	5001	ERVISITSSYNTHETIC_02_Q2.sashdat	132368
-rwxrwxrwx	sas	5001	simulated_admissions_scoring.sas7bdat	262144
-rwxrwxrwx	sas	5001	ALTITUDEEXPRESS.sashdat	3303792
-rwxrwxrwx	sas	5001	lg_attributes.sas7bdat	131072
-rwxrwxrwx	sas	5001	lookingglass_forecast.sas7bdat	12189696

```
/* Run the Summary action */
proc cas;
    simple.summary
     table={caslib="public" name= "Orion_Profit"
            groupBy="Product_Category"}
     inputs={"Profit"}
     subset={"sum"};
quit;
```

Results from simple.summary

Product_Category=Assorted Sports Articles

Descriptive Statistics for ORION_PROFIT

Column	Sum
Profit	834171.16

Results from simple.summary

Product_Category=Children Sports

Descriptive Statistics for ORION_PROFIT

Column	Sum
Profit	175236.29

Results from simple.summary

Product_Category=Clothes

Descriptive Statistics for ORION_PROFIT

Column	Sum
Profit	722265.005

Two examples are shown here.
The fileInfo action displays information about loadable files in directory, object storage bucket, or other source file location.
The Summary action creates summary statistics.

4. CAS Action Result Variables

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

CAS Action Result Variables

Base SAS vs CASL

CASL Variable

```
*** Write SAS Library Info to a Table ***;

proc datasets lib=sashelp noprint;
   contents data=_all_ out=tabInf;
run;
```

```
*** Write CAS Library Info to a CASL Variable ***;

proc cas;
 table.tableinfo result=tabInf / caslib="public";
run;
```

CAS action results can be redirected from the Results (Listing) pane to CASL variables. This is similar to redirecting PROC output to datasets in base SAS.

CAS Action Result Variables

TableInfo Action

```
proc cas;
  table.tableinfo result=tabInf / caslib="public";

    describe tabInf;

quit;
```

SAS Log

```
dictionary ( 1 entries, 1 used);

 [TableInfo] Table ( [19] Rows [25] columns
Column Names:
 [1] Name            [                ] (varchar)
 [2] Label           [                ] (varchar)
 [3] Rows            [                ] (int64)
 [4] Columns         [                ] (int64)
 [5] IndexedColumns  [Indexed Columns ] (int64)
 [6] Encoding        [                ] (char)
 [7] CreateTimeFormatted [Created         ] (varchar)
 [8] ModTimeFormatted [Last Modified   ] (varchar)
 [9] AccessTimeFormatted [Last Accessed   ] (varchar)
 [10] JavaCharSet      [Character Set   ] (char)
 [11] CreateTime       [                ] (double)
 [12] ModTime          [                ] (double)
 [13] AccessTime       [                ] (double)
 [14] Global           [                ] (int32)
 [15] Repeated         [                ] (int32)
 [16] View             [                ] (int32)
 …
```

Here we describe the CASL variable created from the result of a tableInfo action.
Remember that this CAS action describes the tables in a CASLib.
The CASL variable created here, tabInf, is a dictionary with only one entry.
That entry is a table.
This is new. Only the CAS action result option can create a table CASL variable.
The table named TableInfo contains all of the columns we'd expect from a tableInfo action, name, label, # of rows, …

CAS Action Result Variables

TableInfo Action

```
proc cas;
  table.tableinfo result=tabInf / caslib="public";

    print tabInf.TableInfo;

quit;
```

Table Name	Label	Number of Rows	Number of Columns	Number of Indexed Columns	NLS encoding
LOOKINGGLASS_FORECAST		53999	23	0	utf-8
LG_ATTRIBUTES		918	4	0	utf-8
HOME_EQUITY		5960	18	0	utf-8
VS_BANK		1060038	54	0	utf-8
VS_BANK_SMALL		1060038	17	0	utf-8
ER_VISITS_SYNTHETIC_BIASED_BASIC		10000	22	0	utf-8
ERVISITSSYNTHETIC_01_Q1		500	27	0	utf-8
ERVISITSSYNTHETIC_02_Q2		500	27	0	utf-8
TITANIC3_ID		1309	15	0	utf-8
COMMSDATA		56557	128	0	utf-8
ALTITUDEEXPRESS		10000	23	0	utf-8
PARKS_S		56	6	0	utf-8

Here we see a subset of the tableIno result CASL variable.
We use Dictionary.Table Name notation to reference the table.
In the result CASL variable, we see information about the tables in public CASlib.

CAS Action Result Variables

TableInfo Action

```
proc cas;
  table.tableinfo result=tabInf / caslib="public";

    print tabInf.TableInfo[2,"Name"];

quit;
```

SAS Log

```
LOOKINGGLASS_FORECAST
```

We can also reference table values directly using Dictionary.Table name [row, column] notation.
Here we ask for the second row's name field value.
Hopefully, you are starting to see how we can use CASL variables in this way to drive data driven processing.

5. Load all Files in a Directory to CAS

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

Load all Files in a Directory to CAS

SAS Macro

```
cas mysess sessopts=(metrics=true);

%macro import_csv_dir(path);

    %let rc = %sysfunc(filename(mydir, &path));
    %let did = %sysfunc(dopen(&mydir));
    %let memcnt = %sysfunc(dnum(&did));

    %do i = 1 %to &memcnt;
        %let name = %qsysfunc(dread(&did, &i));

        %let dsn_name = %qscan(&name, 1, .);

        proc casutil;
            load file="&path/&name" casout="&dsn_name"
            outcaslib="casuser"     importoptions=(filetype="CSV")
            replace ;
        quit;

    %end;

    %let rc = %sysfunc(dclose(&did));

%mend import_csv_dir;

%import_csv_dir(/home/student/Courses/DDPWC);
```

Create a filename to the directory and get the number of files inside.

Iterate over the files and load any CSVs

We have all of the pieces we need to drive CASL processing with data now.
Before we go there though, we'll look at how this can be done with SAS Macro.
It is important to note that all the classic functionality our clients are used to still functions in SAS Viya.
So we can choose to use CASL or stick to the SAS Macro we know.
The code uses a series of %sysfunc functions to create a SAS filename to a directory, open it, and count the number of files within.
Then the code loops over the files, reading them one by one and loading them to CAS.
The code assumes all of the files are CSV.

Load all Files in a Directory to CAS

CASL

Create a caslib to the directory

```
caslib mylib path="/home/student/Courses/DDPWC";

proc cas;
  table.fileinfo result=filesToLoad  / caslib="mylib";

  do i = 1 to dim(filesToLoad.FileInfo);
      fname = filesToLoad.FileInfo[i,'Name'];
      table.loadtable /
                caslib="mylib"
                path=fname;
  end;
quit;
```

Iterate over the files and load any eligible files

Here we see the equivalent code in CASL.
First we create a CASLib to the directory.
We run a fileInfo CAS action and direct the results to the filesToLoad CASL variable. It will be a dictionary with a nested table as before.
We iterate of the number of eligible files within the directory using the DIM function.
We assign fname to the table name of the result table row for the iteration.
Then we use the loadTable action to load the table.

6. Use a Parameter Table to Drive Processing

Copyright © SAS Institute Inc. All rights reserved.

sas innovate 2026

Use a Parameter Table to Drive Processing

CASL

Read a Parameter Value from a Table

```
proc cas;
    table.fetch result=OrionParm /
        table={caslib="mylib" name="ParameterTable"
               where="process='Orion_Profit'"};


    whereClause = OrionParm.Fetch[1, 'Condition'];


    print whereClause;

    simple.summary
        table={caslib="Public" name="Orion_Profit"
               groupBy="Product_Category"
               where=whereClause}
        inputs={"Profit"}
        subset={"sum"};
quit;
```

Use the Parameter in a CAS Action

MYLIB.PARAMETERTABLE

#	Process	Condition
1	Orion_Profit	Product_Line = 'Sports'
2	Nightly_Star	Time > '00:00:00't
3	Orders_Dashb	Date > '31MAR2026'd

Finally, we show an example of driving processing from a parameter table.
Here, we have a table with various conditions listed for various processes.
We can grab these conditions with a fetch action and insert them directly into CAS action code as well as any CASL conditions.
In this particular example, the Orion_Profit report expects a WHERE clause condition to use in a summary report.
We first fetch the row where the process field equals “Orion_Profit.”
Then we format the Condition by concatenating quotation marks on it and apply it in our summary CAS action.
We can also use this technique to provide source table names, target table names, groupBY clauses, etc.
We can also use it to feed Boolean values to drive conditional processing like we showed in the first example (aggreg8).

Data Driven Programming with CAS in SAS® Viya®

sas innovate 2026

Copyright © SAS Institute Inc. All rights reserved.